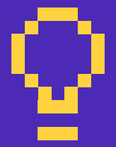
Tuwaiq CLUB

أكاديمية طويق
Tuwaiq Academy

KFUPM
Student Affairs



DAY 3

Serialization & validation

ModelSerializer, function views, and clear input errors

Tuesday, October 27, 2026

Your five-day journey

Yesterday: CRUD by hand. Today: consistent JSON and validated input.

01

Foundation

Project, Task model, migrations, admin

DONE

02

Core API

CRUD with function views

DONE

03

Serialization & validation

Serializers, rules, clear field errors

TODAY

04

Security & configuration

Tokens, ownership, environment

05

Publish

Deploy, verify, README, live demo

Today's session

Learn the serializer, practice the rules, then apply them to the Task API

Explain + live demo
~60 min

Exercise
~60 min

Break
short

Project
~60 min

Exit check
end of day

Priority rule: finish the exercise core, then move to the project. Stretch work can wait.

Learning outcomes

By the end of today you can...

01 Replace manual object conversion with ModelSerializer

02 Use serializers inside the existing function views

03 Make PUT and PATCH behave correctly with partial

04 Reject invalid input with clear field-level errors

The serializer: translator and gatekeeper

days/day-03/example/serializers.py

```
class MovieSerializer(serializers.ModelSerializer):
    director_name = serializers.ReadOnlyField(source="director.name")
    added_by = serializers.ReadOnlyField(source="added_by.username")

    class Meta:
        model = Movie
        fields = ["id", "title", "genre", "release_date",
                  "director", "director_name", "added_by", "created_at"]
        read_only_fields = ["id", "created_at"]
```

The client sends director as an id; director_name and added_by are read-only.

Day 2 vs Day 3: the same view

The URL and ORM query stay the same; the conversion is replaced

Day 2: by hand

```
@api_view(["GET"])
def list_movies(request):
    movies = Movie.objects.all()
    data = [movie_to_dict(m)
            for m in movies]
    return Response(data)
```

Day 3: serializer

```
@api_view(["GET"])
def list_movies(request):
    movies = Movie.objects.all()
    s = MovieSerializer(
        movies, many=True)
    return Response(s.data)
```

One model object -> one dict. many=True -> a list of dictionaries.

Serializers inside function views

The serializer validates input before save() can touch the database

```
@api_view(["POST"])
def create_task(request):
    serializer = TaskSerializer(data=request.data)
    if not serializer.is_valid():
        return Response(serializer.errors, status=400)
    serializer.save(owner=request.user)
    return Response(serializer.data, status=201)
```

data= supplies input. is_valid() checks it. errors explains failure. save() persists it.

Updating with a serializer

Pass the existing instance, incoming data, and the PATCH flag

```
@api_view(["PUT", "PATCH"])
def update_task(request, pk):
    task = get_object_or_404(Task, pk=pk)
    serializer = TaskSerializer(
        task, data=request.data, partial=request.method == "PATCH")
    if not serializer.is_valid():
        return Response(serializer.errors, status=400)
    serializer.save()
    return Response(serializer.data)
```

The same five URLs still work. Delete stays unchanged: no JSON conversion is needed.

One POST request, step by step

POST /api/tasks/create/ -> create_task(request)

01 TaskSerializer(data=request.data) receives the input

02 is_valid() runs built-in, field-level and object-level rules

03 Invalid: return 400 + serializer.errors; save nothing

04 Valid: save(owner=request.user) inserts the row

05 Return 201 + serializer.data, including server-controlled fields

The serializer is the translator and the gatekeeper.

PUT and PATCH

The serializer enforces the difference with partial=False or partial=True

PATCH · partial update

Send only the field that changes. Other fields stay as they are.

```
{"status": "DONE"}   → 200
```

PUT · full replacement

Send the complete object. Missing required fields return 400.

```
{"status": "DONE"}   → 400
```

A partial body is fine for PATCH. PUT needs the complete object.

The three validation levels

Keep validation close to the serializer; declare each rule once

Level	Where	Example / error key
Built-in	`Model field rules`	Required, type, choices, unique ISBN, valid author
Field-level	`validate_<field>(value)`	One field; error appears under that field
Object-level	`validate(self, attrs)`	A rule combining fields; non_field_errors

extra_kwargs customizes built-in messages. Raise ValidationError to reject input.

Validation belongs to the serializer

Built-in rules handle common problems. Custom rules express the program's rules.

Built-in
required fields · wrong types · unique ISBN · unknown author

Clearer messages
extra_kwargs → "Title cannot be blank."

Custom rules
validate_<field> · self.instance tells create from update

Exercise rule
published_year cannot be later than this year

Project rule
due_date cannot be earlier than today on create

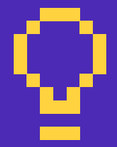
Invalid input returns 400 with field-level messages and no database change.

A field-level validation rule

Project P6: a past due date is rejected on create, while existing tasks stay editable

```
def validate_due_date(self, value):
    if (self.instance is None
            and value is not None
            and value < timezone.localdate()):
        raise serializers.ValidationError(
            "Due date cannot be earlier than today.")
    return value
```

self.instance is None on create; it is the existing object on update.



LIVE DEMO

The same API, clearer errors

5 min: keep the URLs; show what changes when validation runs

Blank title	Bad status	PUT vs PATCH	Valid create
400 + message	400, no save	400 vs 200	201 + JSON

Look at both the response and whether the database changed.

HOUR 2 · EXERCISE

Book API with a serializer

Replace manual conversion, then add the library rules

Core	Time	Stretch
Tasks 1-3	about 50 min	Task 4

Start in days/day-03/exercise/. The five book URLs are unchanged.

Exercise: Tasks 1-3 and stretch

catalog/serializers.py and catalog/views.py

Task	What you build
1	BookSerializer: 8 keys; author_name read-only; id and created_at read-only
2	List + retrieve use .data; create validates + saves; update uses partial
3a	Blank title: "Title cannot be blank." via extra_kwargs
3b	Future published_year: "Published year cannot be in the future."
4 stretch	validate(): no duplicate title per author (case-insensitive); exclude self

Given: AuthorSerializer, the URLs, and delete_book. Complete the self-check.

BREAK

Short break

After the break: apply the same pattern to the Task API

HOUR 3 · PROJECT

Validate the Task API

TaskSerializer replaces manual conversion and repeated checks

Commit Day 2	Checkpoint	P1-P6	10 requests
save your work	day-03	P7 is stretch	verify + push

Keep the same URLs. Per-user isolation arrives on Day 4.

Day 3 project increment

tasks/serializers.py and tasks/views.py; P1-P6 core, P7 stretch

TODO	What you build
P1	8 fields; id, owner, created_at and updated_at read-only
P2	List and retrieve return TaskSerializer(...).data
P3	Create: is_valid() -> 400, or save(owner=request.user) -> 201
P4	Update: partial for PATCH; invalid input -> 400
P5	Title errors: "Title is required." / "Title cannot be blank."
P6	validate_due_date: not before today, on create only
P7 stretch	Show owner as username instead of id

Project checklist: requests 1-5

Log in as alice through /api-auth/login/; POSTs use /api/tasks/create/

#	Request / input	Expected
1	GET /api/tasks/	200; same JSON shape
2	POST: title + owner = 999	201; owner is alice
3	POST: title = " "	400; Title cannot be blank.
4	POST: description, no title	400; Title is required.
5	POST: title = "X", status = "FINISHED"	400; error on status

Invalid input returns field errors and does not create a row.

Project checklist: requests 6-10

POSTs use /api/tasks/create/; use the created id for update and delete

#	Request / input	Expected
6	POST: title + due_date = "2020-01-01"	400; due date earlier than today
7	POST: title + due_date = "tomorrow"	400; invalid date format
8	PUT /api/tasks/{id}/update/ {"status": "DONE"}	400; title is required
9	PATCH /api/tasks/{id}/update/ {"status": "DONE"}	200; only status changes
10	DELETE /api/tasks/{id}/delete/	204

Logged-in users still see all tasks. Day 4 adds isolation.

Day 3 exit check

Ready for the next day when every check passes

01 Serializers replace manual dicts; delete remains unchanged

02 id, owner and timestamps are read-only; owner is set on save

03 Missing/blank title and invalid status return 400 with field errors

04 Past due date on create -> 400; malformed date -> 400

05 PUT needs a title; PATCH accepts one field

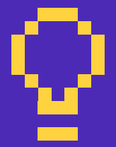
06 All 10 checklist requests pass; the commit is pushed

Next: tokens, ownership, and configuration for deployment

Tuwaiq CLUB

أكاديمية طويق
Tuwaiq Academy

KFUPM
Student Affairs



Day 3 complete

The Task API now converts data consistently and rejects invalid input.

Next: token authentication, per-user ownership, and environment settings